

## EJECUCIÓN ESPERADA DEL COMPILADOR

program.decaf

decaf.tac

generated\_mips.asm

Programa en decaf

Código intermedio

Código en MIPS

```
class TAC2 {
    void main () {
        int var1;
        int var2;
        int var3;

        if (var1 < var2)
        {
            var3 = var1;
        } else {
            var3 = var2;
        }

        var3 = var3 *
    }
}

main:
BeginFunc 8;
_t0 = var1<var2;
Ifz _t0 Goto _L0;
var3 = var1;
Goto _L1;
_L0:
var3 = var2;
_L1:
var3 = var3*var3;
EndFunc;

# auto generated MIPS file.
# VaaDecaf, a Decaf compiler made with Java, ANTLR
# and the Vaadin framework

.text
.globl main
main:
    lw $t1, var1_main      # ld data var1_main
    lw $t2, var2_main      # ld data var2_main
    slt $t0, $t1, $t2
    blez $t0, _L0
    lw $t2, var1_main      # ld data var1_main
    move $s0, $t2          # str data
    sw $s0, var3_main
    b _L1
_L0:
    lw $t2, var2_main      # ld data var2_main
    move $s1, $t2          # str data
    sw $s1, var3_main
_L1:
    lw $t2, var3_main      # ld data var3_main
    lw $t1, var3_main      # ld data var3_main
    mult $t2, $t1
    mflo $s2
    sw $s2, var3_main      # str data

# ----- Exit -----
li $v0, 10
syscall

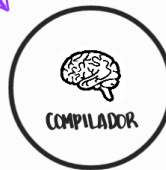
# ----- data section -----
.data
var1_main: .word 0
var2_main: .word 0
var3_main: .word 0
```

Input  
Programa Fuente



Output  
Código de  
3 direcciones

Input  
Código 3  
direcciones



Output  
Código en bajo  
nivel (assembler)

# program.decaf

## Programa en decaf

```
class Factorial {
    int factorial(int n) {
        int ans;
        int i;

        ans = 1;
        i = n;

        while (i > 1) {
            ans = ans * i;
            i = i-1;
        }

        print(ans);
        return ans;
    }

    void main() {
        factorial(5);
    }
}
```

# decaf.tac

## Código intermedio

```
_factorial:
    BeginFunc 16;
    ans = 1;
    i = n;

    StartWhile:

    _L0:
    _t0 = 1<i;
    Ifz _t0 Goto _L1;
    ans = ans*i;
    i = i-1;
    Goto _L0;

    _L1:
    PushParam ans;
    LCall print;
    PopParams 1;
    _t1 = ans;
    _t2 = ans;
    Return _t1;
    EndFunc;

main:
    BeginFunc 8;
    _t0 = 5;
    PushParam _t0;
    LCall _factorial;
    PopParams 4;
    EndFunc;
```

# generated\_mips.asm

## Código en MIPS

# auto generated MIPS file.  
# VaaDecaf, a Decaf compiler made with Java, ANTLR  
and the Vaadin framework

```
.text
.globl main
main:
    li $t0, 5
    move $a0, $t0
    sw $a0, n_factorial
    jal factorial
    move $s0, $v0

    # ----- Exit -----
    li $v0, 10
    syscall

factorial:
    addi $sp, $sp, -4          # Adjust stack
pointer
    sw $s0, 0($sp)             # Save reg
    addi $sp, $sp, -4          # Adjust stack
pointer
    sw $s1, 0($sp)             # Save reg
    addi $sp, $sp, -4          # Adjust stack
pointer
    sw $s2, 0($sp)             # Save reg
    addi $sp, $sp, -4          # Adjust stack
pointer
    sw $s3, 0($sp)             # Save reg
    li $s0, 1
    sw $s0, ans_factorial      # str data
    lw $t0, n_factorial        # ld data
n_factorial
    move $s1, $t0
    sw $s1, i_factorial        # str data
    _L0:
    li $t1, 1
    lw $t2, i_factorial        # ld data
i_factorial
    slt $t0, $t1, $t2
    blez $t0, _L1
    lw $t2, ans_factorial      # ld data
ans_factorial
    lw $t1, i_factorial        # ld data
i_factorial
    mult $t2, $t1
    mflo $s2
    sw $s2, ans_factorial      # str data
    lw $t1, i_factorial        # ld data
i_factorial
    li $t2, 1
    sub $s3, $t1, $t2
    sw $s3, i_factorial        # str data
    b _L0
    _L1:
    li $v0, 1                  # Print call
    lw $a0, ans_factorial
    syscall
    lw $t0, ans_factorial      # ld data
ans_factorial
    move $t1, $t0
    lw $t3, ans_factorial      # ld data
ans_factorial
    move $t2, $t3
    move $v0, $t1              # Return
    lw $s4, 0($sp)             # Restore reg
    addi $sp, $sp, 4           # Adjust stack
pointer
    lw $s3, 0($sp)             # Restore reg
    addi $sp, $sp, 4           # Adjust stack
pointer
    lw $s2, 0($sp)             # Restore reg
    addi $sp, $sp, 4           # Adjust stack
pointer
    lw $s1, 0($sp)             # Restore reg

    addi $sp, $sp, 4           # Adjust stack
pointer
    jr $ra                    # Jump to addr stored in $ra
$ra

# ----- data section -----
.data
n_factorial:                .word 0
ans_factorial:              .word 0
i_factorial:                .word 0
```